In the preceding chapters, the agent interacts with its environment while learning: it tries a price, observes a purchase, and updates its belief. Online learning is natural in digital markets where experimentation is cheap and feedback is instant. In many economically important settings, however, experimentation is impossible or prohibitively costly. A hospital cannot randomly assign treatments to learn optimal dosing. A central bank cannot experiment with interest rate schedules to discover optimal monetary policy. A firm inheriting a decade of transaction logs wants to improve its pricing rule without conducting new experiments during the transition. In each case, the agent has access to a fixed dataset of past decisions and outcomes, collected under some historical policy, and must learn the best possible new policy from this data alone.

This is the problem of *offline reinforcement learning*, also called *batch reinforcement learning*.[1] The agent never queries the environment. All learning happens from a fixed dataset $\mathcal{D} = \{(s_i, a_i, r_i, s_i')\}_{i=1}^n$ collected by some behavioral policy π_b. The goal is to find a policy $\hat{\pi}$ whose value $V^{\hat{\pi}}$ is as close to the optimal V^* as possible.

Online RL algorithms (Q-learning, SARSA, policy gradient methods from Section **??**) can in principle be applied to offline data by treating the dataset as a replay buffer. In practice, this fails catastrophically. The reason is *distributional shift*: the learned policy $\hat{\pi}$ inevitably queries state-action pairs that the behavioral policy π_b never visited, and the Q-function at these unseen pairs is pure extrapolation. Because the Bellman backup propagates these extrapolation errors through the max operator, errors compound geometrically across the planning horizon, producing arbitrarily poor policies.[2]

0.1 The Pessimism Principle

The overestimation failure has a clean theoretical characterization. Consider tabular Q-learning with a fixed dataset. At any state-action pair (s, a) not in the dataset, the empirical Bellman backup is undefined, but the max in $\max_{a'} \hat{Q}(s', a')$ may still select it if the randomly-initialized Q-value happens to be high. With function approximation, the problem is subtler but identical in spirit: the function approximator can assign high

[1] The term "batch RL" was standard in the earlier literature (Ernst et al., 2005; Lange et al., 2012). "Offline RL" became dominant after Levine et al. (2020), who distinguished it from off-policy RL (which still collects new data, just under a different policy than the target). I use "offline RL" throughout.

[2] This failure mode was first demonstrated empirically by Fujimoto et al. (2019), who showed that standard off-policy algorithms (DDPG, SAC) trained purely from a static dataset performed worse than the behavioral policy itself, even when that behavioral policy was a partially-trained, mediocre agent. The gap widened with dataset size, the opposite of what one expects from more data.

values to out-of-distribution inputs without any corrective signal.

The solution, established simultaneously by several groups, is the *pessimism principle*: construct a lower confidence bound on the Q-function and optimize against it. Formally, given a dataset $\mathcal{D}$ and a confidence parameter δ, construct a penalty function $\Gamma(s, a)$ that is large where data coverage is poor, and define the pessimistic Q-function

$$\tilde{Q}(s, a) = \hat{Q}(s, a) - \Gamma(s, a) \tag{1}$$

where $\hat{Q}$ is the standard empirical Bellman solution. The policy $\hat{\pi}(s) = \arg\max_a \tilde{Q}(s, a)$ selects actions that are both high-value *and* well-supported by data.

Definition D1 (Pessimistic Value Iteration, PEVI (Jin et al., 2021))**.** *Given dataset $\mathcal{D}$, penalty function $\Gamma_h(s, a)$ for each stage h, and horizon H, PEVI computes*

$$\hat{Q}_h(s, a) = r_h(s, a) + \hat{P}_h \hat{V}_{h+1}(s, a) - \Gamma_h(s, a) \tag{2}$$

$$\hat{V}_h(s) = \max\{\max_a \hat{Q}_h(s, a),\, 0\} \tag{3}$$

$$\hat{\pi}_h(s) = \arg\max_a \hat{Q}_h(s, a) \tag{4}$$

where $\hat{P}_h$ is the empirical transition operator estimated from $\mathcal{D}$.

Jin et al. (2021) show that with $\Gamma_h(s, a) = c \cdot \sqrt{H^3 / N_h(s, a)}$, where $N_h(s, a)$ counts visits to (s, a) at stage h and c is an absolute constant, PEVI achieves

$$V^* - V^{\hat{\pi}} \leq \tilde{O}\left(\sum_{h=1}^{H} \mathbb{E}_{\pi^*}\left[\sqrt{\frac{1}{N_h(s_h, a_h)}}\right]\right) \tag{5}$$

with high probability. The bound depends on the data coverage at states and actions visited by the *optimal* policy π^*, not the full state-action space. This is the key advantage of pessimism over uniform coverage requirements.

0.1.1 Concentrability and Coverage

The bound in (5) is instance-dependent, scaling with how well π_b covers π^*. The classical way to formalize this is through *concentrability coefficients* (Munos and Szepesvári, 2008).

Definition D2 (Single-policy concentrability)**.** *The single-policy concentrability coeffi-*

cient of π^* with respect to π_b is

$$C^* = \max_{h \in [H]} \left\| \frac{d_h^{\pi^*}}{d_h^{\pi_b}} \right\|_\infty \tag{6}$$

where $d_h^\pi(s, a)$ is the state-action occupancy measure of π at stage h.

When C^* is finite, the optimal policy visits only states and actions that π_b also visits with non-negligible probability, and the $1/\sqrt{N_h(s_h, a_h)}$ terms in (5) remain controlled. Rashidinejad et al. (2021) prove that single-policy concentrability is both necessary and sufficient for offline learning: if $C^* < \infty$, then $O(|\mathcal{S}|H^2 C^*/\epsilon^2)$ samples suffice for an ϵ-optimal policy; if $C^* = \infty$, no algorithm can guarantee suboptimality better than the behavioral policy.

0.1.2 Impossibility Results

Zanette (2021) establish fundamental limits on offline RL by showing that it can be exponentially harder than online RL. Specifically, there exist MDPs with S states and horizon H where online RL finds an ϵ-optimal policy in $\text{poly}(S, H, 1/\epsilon)$ episodes, but any offline algorithm requires $\Omega(2^H)$ samples unless the dataset covers all reachable states. The construction uses a binary tree MDP where the optimal path visits a unique leaf, and any dataset that misses this leaf provides no information about the optimal action at the root.

The practical implication is that offline RL is not a universal replacement for online experimentation. When the behavioral policy is far from optimal, especially in long-horizon problems, offline methods provably cannot recover the optimal policy without exponentially large datasets. Pessimistic algorithms are the best one can do, but they are still fundamentally constrained by what the data contains.

0.2 Algorithms

I present four practical algorithms that instantiate different approaches to the distributional shift problem. All four can be understood as modifications of standard Q-learning (Section **??**) that prevent the agent from overvaluing actions outside the data support.

0.2.1 Fitted Q-Iteration

Fitted Q-Iteration (FQI, Ernst et al., 2005) is the simplest offline RL algorithm, predating the modern pessimism framework. FQI applies the standard Bellman backup iteratively using supervised regression on the fixed dataset, as described in Section **??**. It does not include any explicit pessimism mechanism. When state-action coverage is poor, the max in the Bellman backup selects the highest Q-value among all actions at s', including actions never observed in the data. If the function approximator generalizes poorly at these unseen actions, targets become noisy and biased upward, causing the overestimation cascade. FQI works well when coverage is good but degrades as coverage gaps grow.[3]

0.2.2 Conservative Q-Learning

Conservative Q-Learning (CQL, Kumar et al., 2020) adds an explicit penalty that pushes down Q-values at actions not well-represented in the data. The key idea is to add a regularizer to the Bellman error objective that minimizes Q-values under a broad distribution over actions while maximizing Q-values at the actions actually taken in the dataset.

Definition D3 (Conservative Q-Learning)**.** *CQL modifies the standard Bellman error objective by adding a conservative regularizer. At each iteration, CQL solves*

$$\hat{Q}_{k+1} = \arg\min_{Q} \; \alpha \left(\mathbb{E}_{s\sim\mathcal{D}} \left[\log \sum_{a} \exp Q(s,a) \right] - \mathbb{E}_{(s,a)\sim\mathcal{D}}[Q(s,a)] \right) + \frac{1}{2}\mathbb{E}_{\mathcal{D}} \left[\left(Q(s,a) - \hat{\mathcal{B}}^{\pi_k} \hat{Q}_k(s,a) \right)^2 \right] \tag{7}$$

where $\alpha > 0$ is a hyperparameter controlling the degree of conservatism, and $\hat{\mathcal{B}}^{\pi_k}$ is the empirical Bellman operator.

The first term in the regularizer, $\log \sum_a \exp Q(s,a)$, is a soft maximum over all actions, pushing down Q-values uniformly. The second term, $\mathbb{E}_{\mathcal{D}}[Q(s,a)]$, pulls Q-values back up at the data actions. The net effect is a penalty on Q-values for actions that appear infrequently relative to their softmax contribution. Kumar et al. (2020) prove that the resulting Q-function is a pointwise lower bound on the true Q-function (Theorem 3.2), making it a concrete instantiation of the pessimism principle from (1) with an implicit, data-adaptive penalty Γ.

[3] Munos and Szepesvári (2008) prove finite-time error bounds for FQI under approximate Bellman completeness and all-policy concentrability. Both assumptions are strong, and violation of either leads to divergence in practice.

0.2.3 Implicit Q-Learning

Implicit Q-Learning (IQL, Kostrikov et al., 2022) avoids querying Q-values at unseen actions entirely. Instead of computing $\max_{a'} Q(s', a')$ in the Bellman backup (which requires evaluating Q at potentially out-of-distribution actions), IQL learns a separate value function $V(s)$ that approximates the in-sample maximum through *expectile regression*.

Definition D4 (Implicit Q-Learning)**.** *IQL maintains three functions: $Q_\theta(s, a)$, $V_\psi(s)$, and a policy $\pi_\phi(a|s)$. The value function is trained via expectile regression*

$$L_V(\psi) = \mathbb{E}_{(s,a)\sim\mathcal{D}} \left[L_2^\tau (Q_{\bar{\theta}}(s, a) - V_\psi(s)) \right] \tag{8}$$

where $L_2^\tau(u) = |\tau - \mathbb{1}\{u < 0\}| \cdot u^2$ is the asymmetric squared loss with expectile parameter $\tau \in (0.5, 1)$, and $Q_{\bar{\theta}}$ uses a target network. The Q-function is trained with V_ψ as the continuation value

$$L_Q(\theta) = \mathbb{E}_{(s,a,r,s')\sim\mathcal{D}} \left[(r + \gamma V_\psi(s') - Q_\theta(s, a))^2 \right] \tag{9}$$

The expectile parameter τ controls the degree of optimism within the data support. As $\tau \to 1$, the expectile converges to the in-sample maximum; as $\tau \to 0.5$, it converges to the in-sample mean. Setting $\tau = 0.7$ balances exploiting the best observed actions against the noise in finite samples. The critical property is that the Q-function is never evaluated at actions outside the dataset: the max operation is implicit in the expectile regression of V.

0.2.4 Batch-Constrained Q-Learning

Batch-Constrained Q-Learning (BCQ, Fujimoto et al., 2019) takes a different approach: rather than modifying the Q-function objective, it restricts the policy to actions similar to those in the dataset. BCQ first learns a generative model $G_\omega(s)$ of the behavioral policy, then constrains the policy to only select actions with high likelihood under G_ω.

Definition D5 (Batch-Constrained Q-Learning)**.** *BCQ learns a behavioral model $G_\omega(a|s)$ from the dataset via maximum likelihood, and constrains action selection*

$$\hat{\pi}(s) = \arg \max_{a: G_\omega(a|s) \geq \tau \cdot \max_{a'} G_\omega(a'|s)} Q_\theta(s, a) \tag{10}$$

where $\tau \in (0, 1]$ is a threshold parameter. The Q-function is trained with standard Bellman backups, but the max in the target computation is also constrained to the feasible action set.

BCQ's constraint is hard rather than soft: actions with behavioral probability below τ times the most likely action are simply excluded from consideration. This prevents the Q-function from ever being queried at truly out-of-distribution actions, addressing the distributional shift problem at the policy level rather than the value function level. The tradeoff is that BCQ's performance is bounded by the quality of actions in the dataset. If the behavioral policy never takes the optimal action at some state, BCQ cannot discover it regardless of sample size.

0.3 Trajectory Models and Return-Conditioned Supervised Learning

Pessimism-based offline RL keeps the value function as the central object, with the policy emerging from a Q-function constructed to be conservative outside the data. An alternative inverts that relationship. Train a supervised or generative model on the logged trajectories themselves, and treat the model as the policy. The agent forecasts what to do, and the forecast is the action. Two members of this family bracket the design space, a transformer-based trajectory model and a stripped-down MLP that isolates the operative ingredient.

0.3.1 Decision Transformer

An offline reinforcement-learning dataset is a collection of trajectories $\tau = (s_0, a_0, r_0, s_1, a_1, r_1, \ldots, s_T, a_T,$ generated by some behavioural policy. A trajectory generative model treats τ as a sequence and trains a generative model on those sequences. The Chen et al. (2021) Decision Transformer represents each trajectory as a flat sequence

$$\tau = (\widehat{R}_0, s_0, a_0, \widehat{R}_1, s_1, a_1, \ldots, \widehat{R}_T, s_T, a_T), \tag{11}$$

where $\widehat{R}_t = \sum_{t' \geq t} r_{t'}$ is the return-to-go from time t onwards. A causally masked GPT-style transformer is trained to predict the next action token given the last K context tokens. At test time, the operator specifies a desired return $R^\star$ and primes the model with $\widehat{R}_0 = R^\star$ and the current state s_0. The model generates the corresponding action,

the environment returns a reward and the next state, the operator decrements $R^\star$ by the realised reward, and the loop repeats. No value function, no policy gradient, no critic appears in training. The conditioning variable plays the role of a behavioural target rather than an estimated value function.

The empirical case rests on D4RL and Atari. On the 1% DQN-replay slice of Atari, the Decision Transformer attains gamer-normalised scores of 267.5 on Breakout against the 211.1 of Conservative Q-Learning, the incumbent offline-RL baseline. On the D4RL locomotion benchmark, the Decision Transformer averages 74.7 across nine task-dataset combinations against CQL's 54.2 and behavioural cloning's 47.7.[4]

0.3.2 Return-Conditioned Supervised Learning

The Decision Transformer combines two ideas, a transformer architecture over trajectory tokens and a return-conditioning protocol at deployment. Emmons et al. (2022) isolate the second from the first. Their return-conditioned supervised learning baseline drops the transformer entirely. A small multilayer perceptron is trained by maximum likelihood to predict the action given the current state and a desired return-to-go,

$$\pi_\theta(a \mid s, \widehat{R}) = \operatorname{softmax}\Big(f_\theta(s, \widehat{R})\Big). \tag{12}$$

At deployment time the same protocol applies. Specify a target return $R^\star$, condition on it, execute the predicted action, decrement $R^\star$ by the realised reward, and repeat. Across the D4RL locomotion suite RvS matches the Decision Transformer on most task-dataset pairs at a small fraction of the parameter count, suggesting that return-conditioning rather than the transformer architecture is the operative ingredient. The result raises a caveat that both methods share. Conditioning on a return that is far above any value observed in the training data is an extrapolation request, and the policy's behaviour out of distribution is not constrained by the training objective.[5]

[4] The same trajectory-as-sequence reframing was independently proposed by Janner et al. (2021), with beam search over tokenised states and actions replacing return conditioning. Subsequent diffusion variants (Janner et al., 2022; Ajay et al., 2023) replace autoregressive generation with iterative denoising of an entire trajectory and dispense with dynamic programming entirely. The methodological move is the same, plan by sampling from a generative model of structured objects rather than by maximising a learned value function.

[5] Brandfonbrener et al. (2022) formalise the failure mode. In stochastic environments, conditioning on a high return induces a policy that selects actions whose value distribution has a high upper tail rather than a high expectation, which is a different optimisation problem. The Decision Transformer and RvS recover the optimal policy only when the environment is near-deterministic or when the target return matches what an optimal policy would actually achieve. The reduction-to-supervised-learning that makes the family attractive comes with this

0.4 Simulation Study: Offline RL for Dynamic Pricing

The simulation study evaluates the four offline RL algorithms on a perishable inventory pricing problem with demand regime switching. A retailer with $I_{\max} = 30$ units of perishable inventory must set prices over $H = 20$ periods. The state (i, d, t) consists of current inventory $i \in \{0, \ldots, 30\}$, demand regime $d \in \{1, 2, 3, 4\}$, and time remaining $t \in \{1, \ldots, 20\}$. The action is a price $p \in \{1, \ldots, 10\}$. Demand follows $Q \sim \text{Poisson}(\lambda_0[d] \cdot e^{-0.15p})$ with base rates $\lambda_0 = (1.5, 3.0, 5.0, 8.0)$, and the reward is $r = p \cdot \min(Q, i)$. Demand regimes follow a 4-state Markov chain with diagonal persistence 0.6. Unsold inventory at the terminal period incurs a spoilage cost of \$2.00 per unit, making clearance pricing valuable near the deadline.[6] The behavioral policy represents a conservative pricing team that always sets the maximum price ($p = 10$) regardless of demand regime, inventory, or time remaining, with probability 0.85 and randomizes uniformly over all prices with probability 0.15.[7] All episodes start at full inventory ($i = 30$). All offline methods train on 500 episodes and are evaluated over 1,000 episodes against the DP optimal policy computed by backward induction.[8] Results are averaged over 20 independent seeds.

FQI achieves 81.2% of the DP optimal, substantially below the behavioral cloning baseline of 88.0% (Table 1). Without any mechanism to control extrapolation error, the $\max_{a'} Q(s', a')$ operator in FQI's Bellman backup selects overestimated Q-values at out-of-distribution actions, and these overestimates compound across 200 iterations of fitted Q-iteration. CQL and IQL both exceed the behavioral baseline, achieving 91.9% and 92.0% respectively.[9] CQL's conservative penalty suppresses Q-values at actions not

stochastic-environment caveat.

[6]The spoilage penalty creates distributional shift. The optimal policy adapts prices to inventory level and time remaining, using lower prices near the deadline when inventory is high. The behavioral policy ignores these state variables and prices at the maximum, so the Q-function at state-adapted pricing actions is extrapolation from sparse data. The \$2.00 penalty is a deliberate design choice: under harsher penalties (e.g., \$10 per unit), all methods collapse to 48–53% of optimal regardless of algorithmic sophistication, confirming that no offline correction can overcome severe distributional shift when the penalty regime amplifies consequences of the behavioral policy's suboptimality.

[7]With 500 episodes (10,000 transitions) on a state-action space of 24,800 pairs, the 85% concentration at price 10 ensures that the behavioral state-action occupancy diverges significantly from the optimal policy's occupancy, while the 15% uniform component provides sparse off-policy coverage.

[8]FQI uses the standard Bellman backup with $\max_{a'} Q(s', a')$, deliberately without a target network, to isolate the overestimation cascade as a pedagogical baseline. Adding target networks to FQI mitigates but does not eliminate extrapolation error. CQL and IQL include target networks following their original implementations (Kumar et al., 2020; Kostrikov et al., 2022); for CQL, target networks proved essential, as the conservative penalty amplifies bootstrap instability without them.

[9]CQL uses $\alpha = 0.1$, the result of a search over $\alpha \in \{5.0, 2.0, 0.5, 0.1\}$. Larger values push Q-values down too aggressively, collapsing the learned policy to the behavioral action at most states; the right α is problem-specific

Table 1: Policy value for each offline RL method, expressed as mean return and percentage of the DP optimal. Standard errors computed over 20 seeds.

Method	Mean Return	% of Optimal
DP Oracle	192.41 ± 0.33	100.0%
BC	169.27 ± 0.60	88.0%
FQI	156.18 ± 1.68	81.2%
CQL	176.73 ± 1.13	91.9%
IQL	176.98 ± 0.56	92.0%
BCQ	169.27 ± 0.60	88.0%

well-represented in the data while preserving relative ordering among data-supported actions, allowing the policy to discover lower prices that clear inventory near the deadline. IQL achieves the same improvement through a different mechanism: by replacing $\max_{a'} Q(s', a')$ with the expectile-regressed value function $V(s')$ as the Bellman continuation, IQL avoids querying Q at unseen actions during training while still extracting a policy that improves on the behavioral at states where the 15% noise component revealed better pricing actions. BCQ matches the behavioral at 88.0%; its action constraint restricts the policy to prices near 10, preventing both overestimation and improvement.[10]

These results validate several theoretical predictions from the preceding sections. FQI's degradation below the behavioral baseline (81.2% versus 88.0%) demonstrates the overestimation cascade described in Section 0.2: without pessimism, the $\max_{a'}$ operator selects overestimated Q-values at out-of-distribution actions, and 200 iterations of bootstrapping compound these errors geometrically (Fujimoto et al., 2019). CQL and IQL both exceeding BC confirms the pessimism principle (Section 0.1): CQL's conservative penalty produces a pointwise lower bound on the true Q-function (Definition D3, Theorem 3.2 of Kumar et al. 2020), while IQL's expectile regression (Definition D4) avoids querying Q at unseen actions entirely (Kostrikov et al., 2022). BCQ matching BC exactly illustrates the action-constraint tradeoff formalized in Definition D5: performance is bounded by the quality of actions in the data, and when the behavioral policy concentrates on a single action, no amount of Q-learning can overcome the constraint.

Figure 1 varies the behavioral noise parameter $\epsilon_b \in \{0.05, 0.3, 0.9\}$. CQL and IQL remain above the BC baseline across all coverage levels, confirming that both pessimism

and can vary by orders of magnitude.

[10] When the behavioral policy concentrates 85% probability at a single action, BCQ's threshold constraint permits only that action at most states, effectively reducing BCQ to behavioral cloning regardless of the learned Q-values.

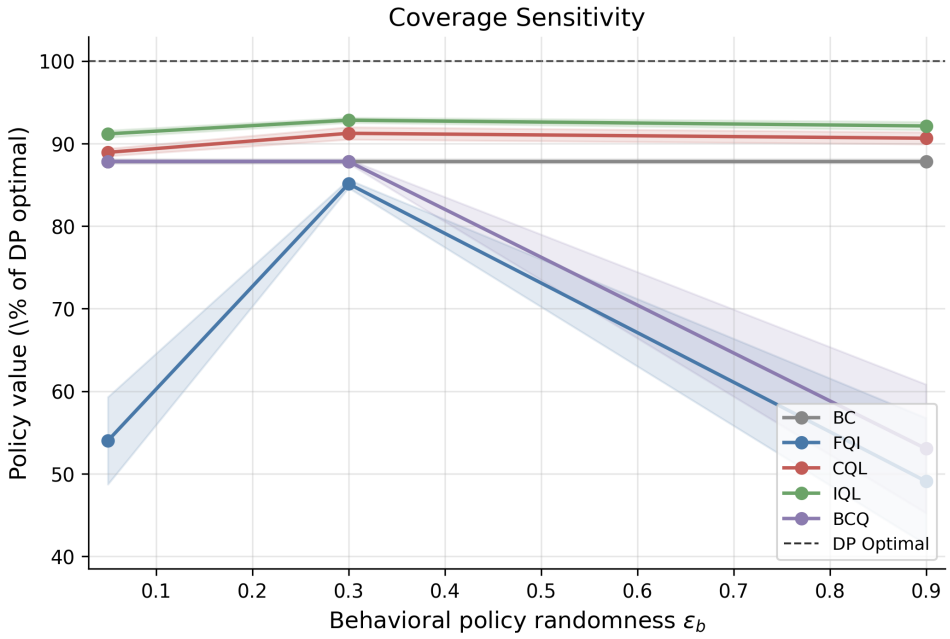

Figure 1: Policy value (as % of DP optimal) versus behavioral policy randomness ϵ_b for four offline RL methods and the behavioral cloning baseline (BC). Higher ϵ_b increases data coverage. FQI peaks at moderate coverage ($\epsilon_b = 0.3$) and collapses at both extremes. BCQ degrades at high ϵ_b as its behavioral constraint becomes vacuous.

mechanisms provide robustness to the data distribution. FQI exhibits non-monotone behavior: it peaks at moderate coverage ($\epsilon_b = 0.3$) where partial exploration controls the overestimation cascade, but collapses at both extremes.[11] BCQ collapses at $\epsilon_b = 0.9$ because a nearly uniform behavioral policy renders the action constraint vacuous, reducing BCQ to unconstrained FQI. These coverage patterns connect directly to the concentrability framework (Definition D2): as ϵ_b decreases, the concentrability coefficient C^* grows because the optimal policy's state-action occupancy diverges from the behavioral policy's, and the $1/\sqrt{N_h(s_h, a_h)}$ terms in (5) become large at states the optimal policy visits. CQL and IQL remain robust because their conservative adjustments scale with data sparsity, while FQI lacks this adaptive correction.

The chapter's object is therefore fixed-data policy improvement when scalar rewards are observed. This is distinct from RLHF, where the reward itself must be inferred from preference comparisons and then interpreted as an object for aggregation, incentives, and welfare. The next chapter takes up that separate problem.

[11] At high ϵ_b, the near-uniform behavioral policy provides Q-function targets across all actions, giving the unconstrained $\max_{a'}$ operator more opportunities to select overestimated values rather than fewer.

References

Anurag Ajay, Yilun Du, Abhishek Gupta, Joshua B. Tenenbaum, Tommi S. Jaakkola, and Pulkit Agrawal. Is conditional generative modeling all you need for decision-making? In *International Conference on Learning Representations*, 2023. arXiv:2211.15657.

David Brandfonbrener, Alberto Bietti, Jacob Buckman, Romain Laroche, and Joan Bruna. When does return-conditioned supervised learning work for offline reinforcement learning? In *Advances in Neural Information Processing Systems*, volume 35, 2022.

Lili Chen, Kevin Lu, Aravind Rajeswaran, Kimin Lee, Aditya Grover, Michael Laskin, Pieter Abbeel, Aravind Srinivas, and Igor Mordatch. Decision transformer: Reinforcement learning via sequence modeling. In *Advances in Neural Information Processing Systems*, volume 34, 2021.

Scott Emmons, Benjamin Eysenbach, Ilya Kostrikov, and Sergey Levine. RvS: What is essential for offline RL via supervised learning? In *International Conference on Learning Representations (ICLR)*, 2022.

Damien Ernst, Pierre Geurts, and Louis Wehenkel. Tree-based batch mode reinforcement learning. *Journal of Machine Learning Research*, 6:503–556, 2005.

Scott Fujimoto, David Meger, and Doina Precup. Off-policy deep reinforcement learning without exploration. In *Proceedings of the 36th International Conference on Machine Learning*, volume 97 of *Proceedings of Machine Learning Research*, pages 2052–2062. PMLR, 2019.

Michael Janner, Qiyang Li, and Sergey Levine. Offline reinforcement learning as one big sequence modeling problem. In *Advances in Neural Information Processing Systems*, volume 34, 2021.

Michael Janner, Yilun Du, Joshua B. Tenenbaum, and Sergey Levine. Planning with diffusion for flexible behavior synthesis. In *International Conference on Machine Learning*, 2022. arXiv:2205.09991.

Ying Jin, Zhuoran Yang, and Zhaoran Wang. Is pessimism provably efficient for offline RL? In *Proceedings of the 38th International Conference on Machine Learning*, volume 139 of *Proceedings of Machine Learning Research*, pages 5084–5096. PMLR, 2021.

Ilya Kostrikov, Ashvin Nair, and Sergey Levine. Offline reinforcement learning with implicit Q-Learning. In *International Conference on Learning Representations*, 2022.

Aviral Kumar, Aurick Zhou, George Tucker, and Sergey Levine. Conservative Q-Learning for offline reinforcement learning. In *Advances in Neural Information Processing Systems*, volume 33, 2020.

Sascha Lange, Thomas Gabel, and Martin Riedmiller. Batch reinforcement learning. *Reinforcement Learning: State-of-the-Art*, pages 45–73, 2012.

Sergey Levine, Aviral Kumar, George Tucker, and Justin Fu. Offline reinforcement learning: Tutorial, review, and perspectives on open problems. *arXiv preprint arXiv:2005.01643*, 2020.

Rémi Munos and Csaba Szepesvári. Finite-time bounds for fitted value iteration. *Journal of Machine Learning Research*, 9:815–857, 2008.

Paria Rashidinejad, Banghua Zhu, Cong Ma, Jiantao Jiao, and Stuart Russell. Bridging offline reinforcement learning and imitation learning: A tale of pessimism. In *Advances in Neural Information Processing Systems*, volume 34, 2021.

Andrea Zanette. Exponential lower bounds for batch reinforcement learning: Batch RL can be exponentially harder than online RL. In *Proceedings of the 38th International Conference on Machine Learning*, volume 139 of *Proceedings of Machine Learning Research*. PMLR, 2021.